

Beyond Half Twists: Joint Topology and Adaptive Routing Design for Asymmetric 3D Torus

Fan Zhengyang, Xu Yudong
IIIS, Tsinghua University

I. INTRODUCTION

3D topologies such as the 3D torus are well known for their performance and scalability advantages over 2D topologies. However, in real systems, hardware and deployment constraints sometimes require an asymmetric 3D torus. Google Cloud TPU v4, for example, supports $8 \times 4 \times 4$ and $8 \times 8 \times 4$ 3D-torus slices [3]. This asymmetry can introduce imbalanced resource utilization among different directions.

To address this problem, prior work introduced the twisted 3D torus, in which one dimension or two dimensions are twisted. Based on this idea, we further explore other twist parameters that produce shorter diameters and lower average hop counts, and analyze the performance of different twisting schemes. We find that optimizing for shorter paths can reduce shortest-path diversity, which may cause congestion under some routing algorithms. Therefore, we further improve the routing algorithm to alleviate this problem.

In short, we cover the requirements of Topic 1:

- Implement the twisted 3D torus topology in gem5.
- Implement an adaptive deadlock-free routing algorithm.

We further:

- Propose a method for finding better twist parameters with a smaller diameter and lower average distance.
- Improve the routing algorithm to alleviate congestion caused by fewer shortest paths.
- Evaluate different twist parameters and routing algorithms to demonstrate the effectiveness of our methods.

II. ARCHITECTURE

A. Topology: Twisted 3D Torus

We use the $2k \times k \times k$ 3D torus supported by Google Cloud TPU v4 as an example of an asymmetric 3D torus [3].

The problem with an ordinary 3D torus is that packets travel farther on average in the X dimension, so X links can become the bottleneck while links in the short dimensions remain underused. A twisted torus alleviates this problem by changing the endpoints of selected peripheral links.

Definition 1. We define a twisted 3D torus as $T(2k, k, k; p, q)$, and we say it has a shift (p, q) . This means crossing the Y boundary also moves a packet by p in X , while crossing the Z boundary moves it by q in X .

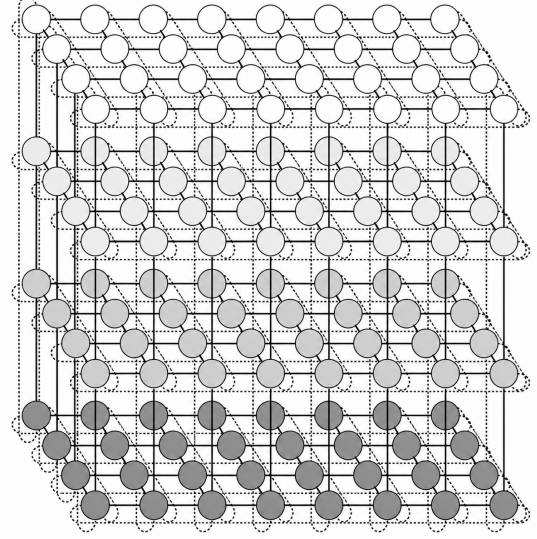


Fig. 1. An unfolded view of the $8 \times 4 \times 4$ 3D torus. Solid lines show regular links, while dashed lines show wraparound connections.

Namely, all links are:

$$(x, y, z) \leftrightarrow (x + 1 \bmod 2k, y, z), \quad (1)$$

$$(x, y, z) \leftrightarrow (x, y + 1, z), \quad y < k - 1, \quad (2)$$

$$(x, k - 1, z) \leftrightarrow (x + p \bmod 2k, 0, z), \quad (3)$$

$$(x, y, z) \leftrightarrow (x, y, z + 1), \quad z < k - 1, \quad (4)$$

$$(x, y, k - 1) \leftrightarrow (x + q \bmod 2k, y, 0). \quad (5)$$

Cámara et al. introduced two members of this family for prismatic $2k \times k \times k$ networks [1]:

Definition 2. A Prismatic Twisted Torus (PTT) twists one short dimension by half the X circumference, corresponding to $(p, q) = (k, 0)$.

Definition 3. A Prismatic Doubly Twisted Torus (PDTT) applies the same half shift to both short dimensions, corresponding to $(p, q) = (k, k)$.

And obviously, we have:

Definition 4. An ordinary 3D torus is a twisted torus with shift $(p, q) = (0, 0)$.

Prior work motivated PTT and PDTT mainly through more balanced link utilization. Nevertheless, we note that such twists also reduce network distances.

For simplicity, let's consider an $8 \times 4 \times 4$ 3D torus.

For the ordinary 3D torus:

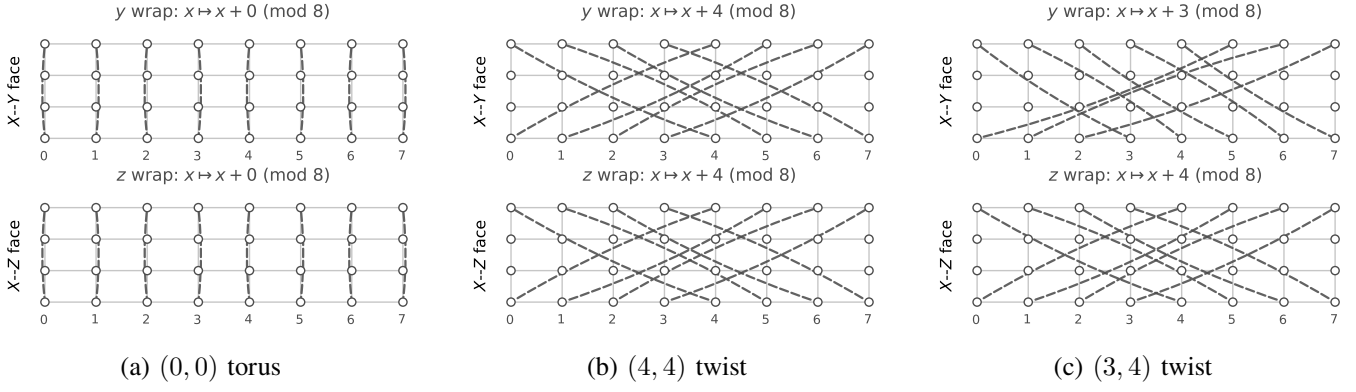


Fig. 2. Boundary-link schematics for the ordinary $8 \times 4 \times 4$ torus, the conventional (4,4) PDTT, and our (3,4) twisted torus. Dashed lines show the eight wraparound links across each boundary; the remaining links are unchanged.

TABLE I
STRUCTURAL COMPARISON OF $8 \times 4 \times 4$ TWIST PARAMETERS.

Twist (p, q)	Diameter	Average shortest distance
(0,0)	8	4.0000
(4,0)	6	3.6250
(4,4)	6	3.4375
(3,4)	5	3.3750

TABLE II
SHORTEST-PATH DIVERSITY IN AN $8 \times 4 \times 4$ NETWORK.

Twist	$\Pr[m(s, t) > 1]$	Mean $m(s, t)$	Maximum
(0,0)	0.9213	81.1969	3360
(4,0)	0.8898	27.9370	360
(4,4)	0.8583	20.0315	360
(3,4)	0.8583	9.7165	60

(1) Diameter: $4 + 2 + 2 = 8$.

(2) Average shortest distance:

To calculate this, let's start from $(0, 0, 0)$ and consider the distances from $(0, 0, 0)$ to all 128 destinations, including itself. The source contributes zero distance.

For an even-sized ring of length n , the sum of shortest distances from one node to all n nodes (including itself) is $n^2/4$. Therefore, in our torus, the total contribution of the X dimension over all 128 destinations is

$$4 \times 4 \times \frac{8^2}{4} = 256,$$

while the contributions of the Y and Z dimensions are each

$$8 \times 4 \times \frac{4^2}{4} = 128.$$

Following the convention of the twisted-torus literature and including the zero-distance source, the average shortest-path distance is thus

$$\bar{d}_{\text{torus}} = \frac{256 + 128 + 128}{128} = \frac{512}{128} = 4.0000.$$

For PTT, PDTT, and our (3,4) twist, the diameter and average distance can be calculated similarly. See Appendix B for calculation details.

The full results are shown in Table I. We can see PTT and PDTT both lead to shorter distance.

This motivates us to ask:

Problem 1. Can we find a twist scheme that further reduces the distance?

For this problem, we have:

Theorem 1. Among all dual-twist schemes (p, q) for a $2k \times k \times k$ torus, the minimum achievable diameter is

$$\min_{p,q} D(T(2k, k, k; p, q)) = \left\lceil \frac{3k}{2} \right\rceil - 1. \quad (6)$$

The twist $(p, q) = (k-1, k)$ attains this minimum.

Proof. See Appendix A.

For the $8 \times 4 \times 4$ example, $k = 4$, so Theorem 1 selects $(p, q) = (3, 4)$ and gives a diameter of $\lceil 3k/2 \rceil - 1 = 5$. Its average shortest-path distance is 3.3750, as shown in Table I.

Among the schemes in Table I, the (3,4) twist has the smallest diameter and average distance. This naturally leads to our second question:

Problem 2. Will the (3,4) twisted 3D torus produce even lower latency?

While the (3,4) twist reduces distance, we find that it also reduces shortest-path diversity. For each ordered source-destination pair (s, t) , let $m(s, t)$ denote the number of distinct shortest paths from s to t . We use BFS with shortest-path counting to compute this value for all $s \neq t$. The results are shown in Table II.

Together with Table I, these results reveal a trade-off: shorter distances tend to come with lower shortest-path diversity. Compared with the (4,4) topology, the (3,4) topology has a 51.5% lower mean path multiplicity, while its maximum multiplicity decreases from 360 to 60. As a result, minimal routing has fewer equal-length alternatives available to bypass congested links. This may cause the (3,4) twist to perform poorly under certain traffic patterns. We will evaluate this

hypothesis in Section IV. To alleviate this issue, we propose the routing algorithm described in Subsection C.

B. AdaptiveRoute

Before introducing our improved routing algorithm designed for (3,4) twist, let's first have a look at our basic adaptive deadlock-free design, called *AdaptiveRoute*.

This routing algorithm uses two channel classes: adaptive VCs and an escape VC. Adaptive VCs carry normal minimal traffic, while the escape VC provides a deadlock-free fallback. Virtual channels provide the logical channel separation required by this design [2].

Let $d(v, t)$ denote the shortest-path distance from router v to destination t . At router v , *AdaptiveRoute* determines the next hop as follows:

Step 1: Minimal-candidate construction. Construct the set of neighboring routers that lie on a shortest path to the destination:

$$M(v, t) = \{u \mid u \text{ is adjacent to } v, d(u, t) = d(v, t) - 1\}. \quad (7)$$

Step 2: Credit-based adaptive selection. Among the candidates in $M(v, t)$, select the output with the largest total number of downstream buffer credits across its adaptive VCs.

Step 3: Escape fallback. If the selected minimal output has no available adaptive VC, transfer the packet to the escape VC and continue routing along the escape path.

Specifically, escape VC follows a spanning-tree route whose channel dependency graph is acyclic. A packet may transition from an adaptive VC to the escape VC, but it is not allowed to switch back to an adaptive VC. Therefore, every packet always has access to a deadlock-free escape path to its destination.

C. FlexRoute

It is easy to tell that *AdaptiveRoute* is a minimal routing algorithm. Recall that, in Subsection A, our (3, 4) twisted 3D torus has significantly fewer shortest paths.

To alleviate the potential congestion caused by this fact, we propose *FlexRoute*. Speaking plainly, it gives each packet one chance to move onto a slightly longer path.

Namely, in addition to $M(v, t)$, *FlexRoute* considers lateral candidates

$$S(v, t) = \{u \mid u \text{ is adjacent to } v, d(u, t) = d(v, t)\}. \quad (8)$$

A lateral hop moves the packet away from a congested shortest path while increasing the total distance by 1. Each packet may use at most one such hop, so a non-escape route is at most one hop longer than the shortest path. This bound prevents livelock but is not by itself a deadlock proof. Deadlock freedom comes from confining lateral hops to the adaptive VC class and retaining the same acyclic escape subnetwork, with transitions allowed only from adaptive VCs to the escape VC.

FlexRoute considers lateral hops only when no minimal output has a free adaptive VC or the available minimal paths are congested. It scores each lateral candidate by jointly considering its two-hop bottleneck credits, onward minimal-path diversity, and VC availability.

```

1 east = ((x + 1) % size_x, y, z)
2 west = ((x - 1) % size_x, y, z)
3
4 if y == size_y - 1 and twist_mode != "none":
5     north = ((x + y_shift) % size_x, 0, z)
6 else:
7     north = (x, (y + 1) % size_y, z)
8 if y == 0 and twist_mode != "none":
9     south = ((x - y_shift) % size_x, size_y - 1, z)
10 else:
11     south = (x, (y - 1) % size_y, z)
12
13 if z == size_z - 1 and twist_mode in ("double", "synth", "custom"):
14     up = ((x + z_shift) % size_x, y, 0)
15 else:
16     up = (x, y, (z + 1) % size_z)
17 if z == 0 and twist_mode in ("double", "synth", "custom"):
18     down = ((x - z_shift) % size_x, y, size_z - 1)
19 else:
20     down = (x, y, (z - 1) % size_z)
21
22 destinations = (east, west, north, south, up, down)

```

Fig. 3. Implementation of ordinary and twisted wrap-around links in *TwistedTorusGraph.py*.

Specifically, suppose a packet is at router v and destined for router t . For each lateral candidate $u \in S(v, t)$:

$$\text{score}(u) = B(u, t) + \alpha |M(u, t)|. \quad (9)$$

Here, $B(u, t)$ is the bottleneck credit, determined by the available credits on link (v, u) and the best minimal continuation from u toward t ; $|M(u, t)|$ is the number of minimal next-hop choices available at u . *FlexRoute* chooses the candidate with the highest score. If no lateral candidate qualifies, it falls back to *AdaptiveRoute*. Further implementation details are provided in Subsection C of the next section.

III. IMPLEMENTATION

A. Topology: Twisted 3D Torus

The topology we implement supports five twist modes:

- none: an ordinary 3D torus with shift $(0, 0)$;
- single: the PTT shift $(k, 0)$;
- double: the PDTT shift (k, k) ;
- synth: our selected shift $(k - 1, k)$;
- custom: any customized shift (p, q) .

We separate the graph definition into *TwistedTorusGraph.py*. For each router, it converts the router ID to coordinates (x, y, z) and returns the destination of its six directed links. The internal links follow Eqs. (1)–(5).

Figure 3 shows the corresponding implementation of the six neighbors and the twisted boundary links.

Finally, *TwistedTorus.py* creates one *gem5 Router* for each coordinate, attaches Ruby controllers through *ExtLinks*, and creates the six directed *IntLinks* returned by the graph definition. The same neighbor calculation is reproduced in *RoutingUnit.cc*, so the C++ routing algorithm uses exactly the topology constructed by Python.

B. AdaptiveRoute

Our implementation of routing algorithm basically follows the algorithm described in Subsection B of the previous section. We introduce it in three parts:

Adaptive routing. We extend `RoutingUnit.cc` to implement the adaptive routing logic. For each destination, we use BFS to compute and cache the shortest distance from every router. Suppose a packet is currently at router v and is destined for router t . The routing unit examines the six directions and returns all minimal candidates. For each candidate output, we sum the downstream credits of all its adaptive VCs. The candidate with the largest total credit count is selected. If multiple candidates have the same maximum count, one of them is selected randomly.

Transition from adaptive VCs to the escape VC. We extend `SwitchAllocator.cc` to perform the transition. When a head flit has not yet obtained an output VC and the output selected by *AdaptiveRoute* has no free adaptive VC, the allocator attempts to switch to the escape VC. Once it does so, it is not allowed to return to an adaptive VC.

Escape routing. We also extend `RoutingUnit.cc` to implement escape routing. A deterministic BFS spanning tree rooted at router 0 is constructed and cached. An escape packet first moves toward its common ancestor with destination and then moves down the tree toward the destination. Because an escape route consists only of upward tree edges followed by downward tree edges, its channel dependency graph is acyclic, so it is deadlock free.

C. FlexRoute

We implement *FlexRoute* by extending the custom routing logic in `RoutingUnit.cc`. While retaining adaptive and escape VC classes of *AdaptiveRoute*, we mainly add two parts.

Candidate construction. For a packet currently at router v and destined for router t , we first construct the minimal set $M(v, t)$ and the lateral set $S(v, t)$ defined in Section II-C. Then we detect congestion. For a candidate next router u , let $C(v, u)$ be the number of credits on the adaptive VCs of link (v, u) . The two-hop bottleneck of candidate u is calculated as

$$B(u, t) = \min\{C(v, u), \max_{w \in M(u, t)} C(u, w)\}. \quad (10)$$

For the minimal candidates, we compute

$$B_M = \max_{u \in M(v, t)} B(u, t). \quad (11)$$

Let F_M indicate whether any minimal output has a free adaptive VC, and let $C_{\max}$ be the total adaptive-VC buffer capacity of one output. For a lateral candidate $u \in S(v, t)$, we define the pressure condition as

$$P(u) = \neg F_M \vee (B_M < \tau C_{\max} \wedge B(u, t) \geq B_M + H), \quad (12)$$

where τ is the configurable credit threshold and H is the configurable credit margin. This means that we seek flex routing when no minimal output has a free adaptive VC (the first term), or when the minimal outputs are congested and a lateral hop can relieve that congestion (the second term).

A lateral candidate $u \in S(v, t)$ is eligible only if the packet still has its one-hop slack, $P(u)$ is true, and the output to u has a free adaptive VC.

Candidate scoring and selection. Recall each eligible lateral candidate is scored as

$$\text{score}(u) = B(u, t) + \alpha |M(u, t)|. \quad (13)$$

The implementation obtains $C(v, u)$ from the downstream credit counters of the output to u , obtains $M(u, t)$ from the cached shortest-distance table, and then evaluates $B(u, t)$ and $|M(u, t)|$ directly according to Eqs. (10) and (13).

To engage randomness, consider

$$E_\epsilon = \{u \mid \text{score}(u) \geq \text{score}_{\max} - \epsilon\}. \quad (14)$$

FlexRoute selects from this set randomly, with probability proportional to $\max\{1, \text{score}(u) + 1\}$. If no lateral candidate is eligible, it invokes *AdaptiveRoute* unchanged.

IV. EVALUATION

We evaluate a 128-router $8 \times 4 \times 4$ network in gem5 Garnet. All experiments inject one-flit control packets through VNet 0. Each virtual network has four VCs: the first three are adaptive VCs and the last is the dedicated escape VC.

For *FlexRoute*, the one-hop slack budget is one, the pressure threshold is $\tau = 100\%$, the credit margin is $H = 2$, the diversity weight is $\alpha = 1$, and the randomized near-best window is $\epsilon = 1$. The following is an example command for running (3, 4) *FlexRoute* under uniform-random traffic:

```
gem5/build/NULL/gem5.opt \
--outdir=m5out/flex \
gem5/configs/example/garnet_synth_traffic.py \
--network=garnet --num-cpus=128 --num-dirs=128 \
--topology=TwistedTorus \
--torus-x=8 --torus-y=4 --torus-z=4 \
--twist-mode=custom --twist-y-shift=3 \
--twist-z-shift=4 --routing-algorithm=2 \
--twist-routing-policy=flex \
--flex-credit-threshold-pct=100 \
--flex-credit-margin=2 \
--flex-diversity-weight=1 --flex-score-epsilon=1 \
--synthetic=uniform_random --injectionrate=0.30 \
--inj-vnet=0 \
--precision=5 --sim-cycles=10000 \
--sys-clock=2GHz \
--ruby-clock=2GHz --link-latency=1 \
--router-latency=1 --link-width-bits=128 \
--vcs-per-vnet=4
```

A. Different Twist Topologies

We first test the effect of our topology by applying the same *AdaptiveRoute* to four twist schemes under uniform-random traffic. Figure 4 shows the reception rate and average packet latency as the injection rate increases.

Before saturation, all three twisted topologies achieve lower latency than the ordinary (0, 0) torus. They also sustain higher reception rates as the offered load increases. In particular, our (3, 4) twist successfully attains the highest peak reception rate and the lowest latency at injection rate 0.05. The detailed results are listed in Table III.

Interestingly, we also compare the theoretical average shortest distance with the measured results, as shown in Table IV. We find that the measured average-hop reductions closely match the theoretical reductions. For example, (3, 4) reduces

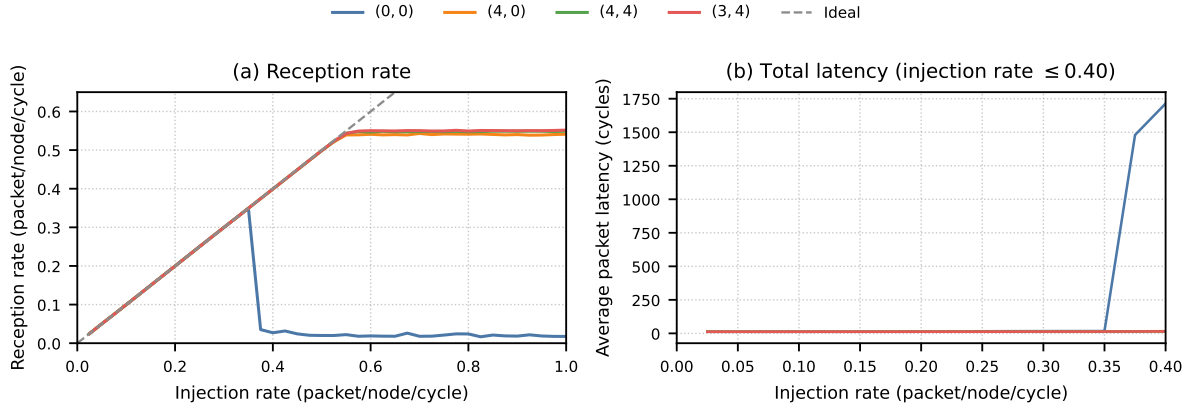


Fig. 4. Performance of different $8 \times 4 \times 4$ twist topologies under uniform-random traffic. All four designs use *AdaptiveRoute*.

the calculated average distance by 15.62% and the measured average hops by 15.89%. This validates our theoretical distance analysis and confirms that its structural improvement is realized by the routing implementation.

B. Routing Comparison

We next test our *AdaptiveRoute* and *FlexRoute* on the $(3, 4)$ twist under uniform-random traffic. We implement deterministic minimal routing in gem5 as the baseline. It always selects the first shortest-path candidate using the fixed direction priority $X^+, X^-, Y^+, Y^-, Z^+, Z^-$.

As shown in Figure 5, *AdaptiveRoute* and *FlexRoute* behave similarly to deterministic minimal routing at low load, but exhibit lower latency while maintaining a similar reception rate as the network approaches saturation. This demonstrates that choosing a direction with more downstream credits can alleviate congestion. Under uniform-random traffic, *FlexRoute* performs similarly to *AdaptiveRoute*. In the next subsection, we show the power of *FlexRoute* under traffic patterns that create more concentrated congestion.

C. FlexRoute Unleashes the Potential of the $(3, 4)$ Twist

In this subsection, we test different settings under various traffic patterns to show that *FlexRoute* unleashes the potential of the $(3, 4)$ twist by increasing path diversity.

C.1 Permutation Traffic: We first test three permutation traffic patterns adopted by the original twisted-torus work [1]: Bit-Complement, Bit-Reversal, and Perfect Shuffle. Figure 6 reports the reception rate and latency of received packets.

TABLE III
UNIFORM-RANDOM PERFORMANCE RESULTS. PEAK RECEPTION RATE IS THE MAXIMUM OBSERVED VALUE OVER THE SWEEP; LATENCY IS MEASURED AT INJECTION RATE 0.05.

Twist	Peak recv. rate	Latency at 0.05
(0, 0)	0.349	13.0863
(4, 0)	0.543	12.3072
(4, 4)	0.549	11.9173
(3, 4)	0.552	11.7939

TABLE IV
COMPARISON BETWEEN THEORETICAL SHORTEST-PATH DISTANCE AND MEASURED HOP COUNT. REDUCTIONS ARE RELATIVE TO THE $(0, 0)$ TORUS.

Twist	Theory dist.	Dist. red.	gem5 hops	Hop red.
(0, 0)	4.0000	–	4.0015	–
(4, 0)	3.6250	9.38%	3.6185	9.57%
(4, 4)	3.4375	14.06%	3.4268	14.36%
(3, 4)	3.3750	15.62%	3.3659	15.89%

The results for Bit-Reversal and Perfect Shuffle are typical. The $(3, 4)$ twist with *AdaptiveRoute* provides competitive reception rate and latency, and *FlexRoute* further improves both metrics around the saturation knee.

Notably, we see a significant reduction of latency in medium-load region. At injection rate 0.50, *FlexRoute* increases reception rate by 5.29% and reduces latency by 53.08% under Bit-Reversal. Under Perfect Shuffle, the corresponding improvements are 2.37% and 56.36%. The reception-rate improvements remain visible up to injection rate 1.0.

Bit-Complement presents a more mixed result. The figure shows that asymmetric topologies such as $(4, 0)$ and $(3, 4)$ perform worse with *AdaptiveRoute*. However, *FlexRoute* successfully improves the reception rate and reduces the latency of $(3, 4)$ to a level comparable to $(4, 4)$. This result shows that $(3, 4)$ does not always perform best because its shortest paths offer fewer choices, but *FlexRoute* can alleviate this drawback by providing additional congestion-aware lateral choices.

To show concretely that lateral hops are indeed in use, we also measure lateral usage. It is near zero at low load for most patterns and generally increases with injection rate. Together with the traffic patterns in the next section, the results show that higher lateral-hop usage generally corresponds to a larger performance gap between *FlexRoute* and *AdaptiveRoute*. This confirms that *FlexRoute* activates its one-hop detours in response to the congestion structure.

These results show that the shorter $(3, 4)$ topology can retain its distance advantage while *FlexRoute* alleviates the congestion caused by its limited minimal-path diversity.

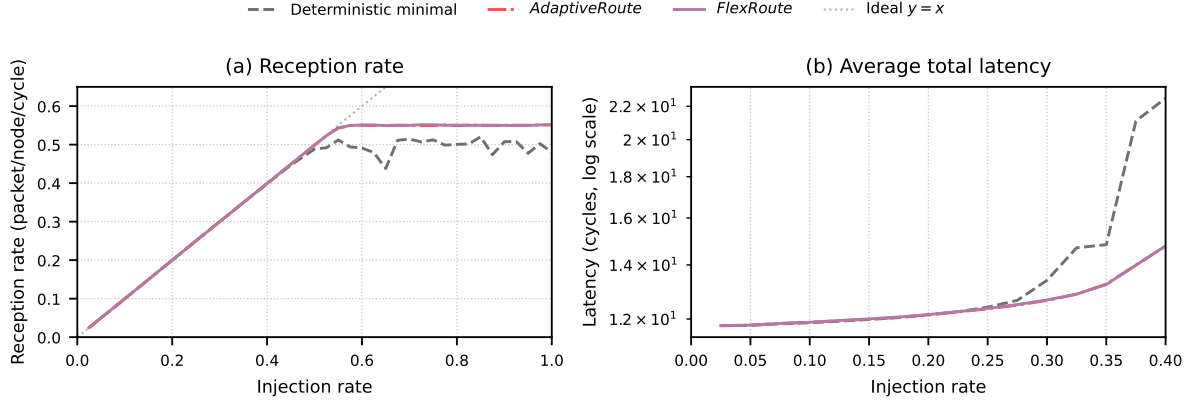


Fig. 5. Comparison of deterministic minimal routing, *AdaptiveRoute*, and *FlexRoute* on the (3, 4) topology under uniform-random traffic. Reception rate is shown over the full injection range, while latency is shown only up to injection rate 0.40.

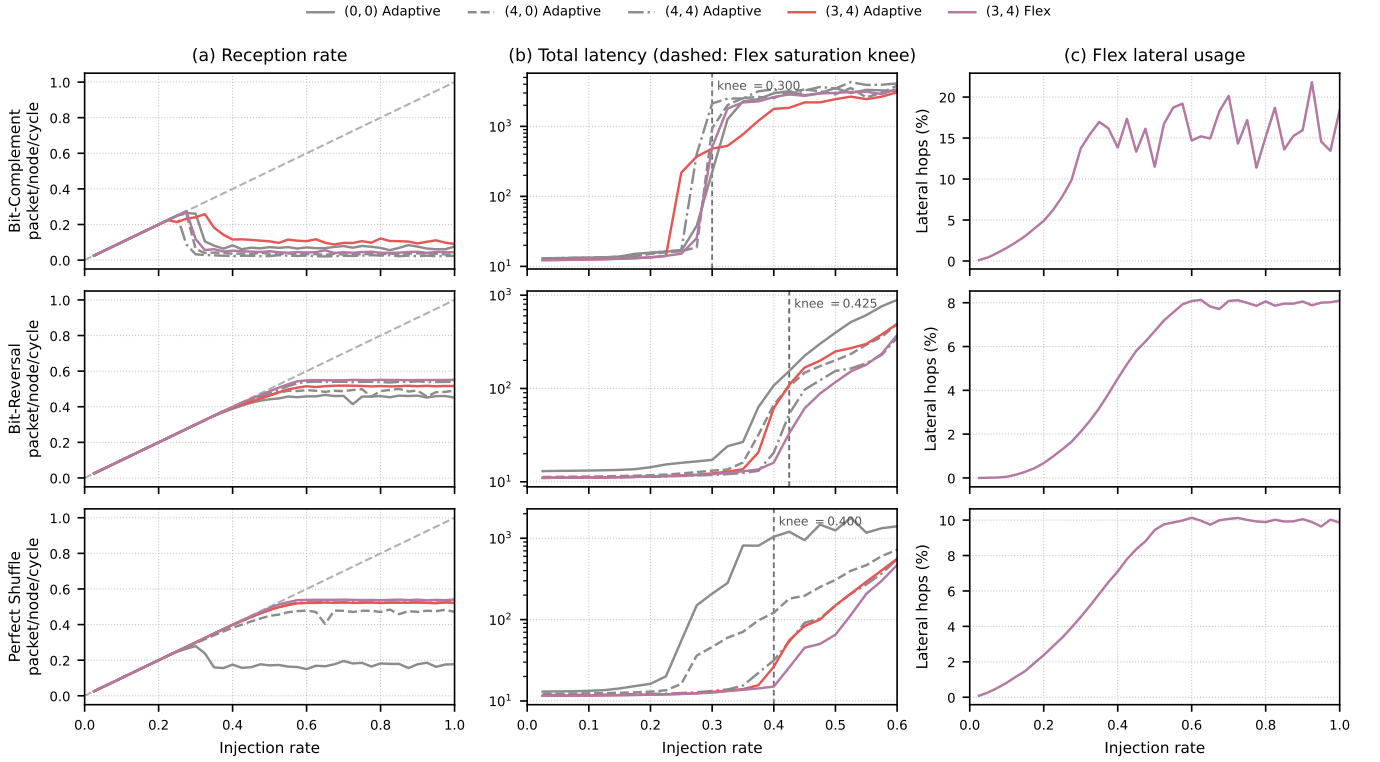


Fig. 6. Comparison of all five designs under the three permutation traffic patterns used in prior twisted-torus evaluation. Vertical dashed lines in the latency panels mark the estimated saturation knee of (3, 4) FlexRoute.

C.2 Directional Traffic: To further evaluate our design under various traffic patterns, we use another group of patterns, including Diagonal-3D, X-Tornado, and Y-Tornado. Diagonal-3D sends each packet to the opposite point in all three dimensions. X-Tornado and Y-Tornado apply a fixed directional offset in the corresponding dimension. These patterns expose directional bottlenecks that uniform-random traffic can hide. The results are shown in Figure 7.

All twisted topologies and routing schemes behave similarly under Y-Tornado and perform worse than (0, 0). Since the

Y dimension has only four routers, the tornado offset is one hop and is therefore equivalent to neighbor traffic along Y . In the ordinary torus, this move is always a direct Y hop. For sources at the Y boundary, however, a twisted wraparound link also changes X : by four routers for (4, 0) and (4, 4), and by three routers for (3, 4). Reaching the same- X destination then requires compensating movement, explaining why the twisted designs do not help this pattern. FlexRoute also provides little benefit; its low lateral usage supports this explanation.

X-Tornado is where FlexRoute exhibits its clearest perfor-

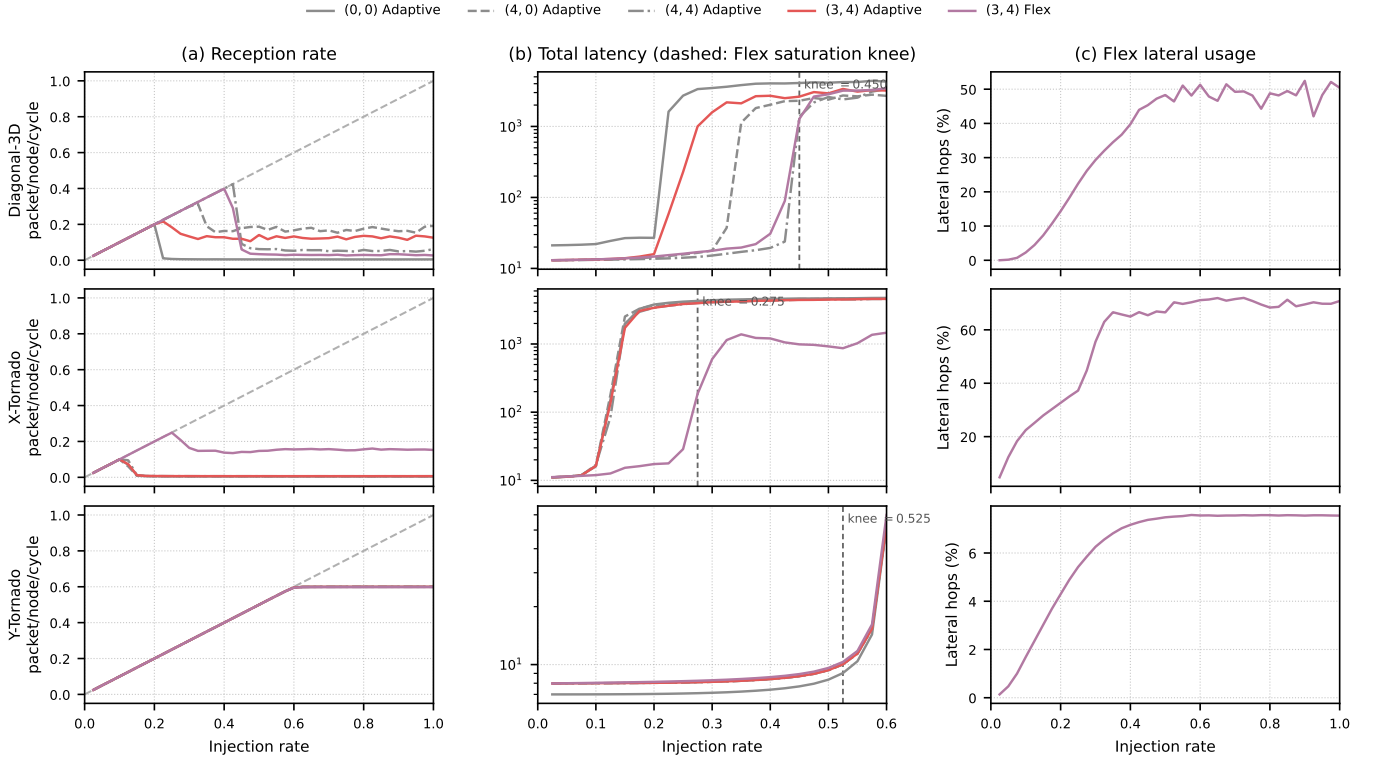


Fig. 7. Comparison of all five designs under Diagonal-3D, X-Tornado, and Y-Tornado traffic. Vertical dashed lines in the latency panels mark the estimated saturation knee of (3, 4) FlexRoute.

mance improvement. We can see it sustains a substantially higher reception rate and lower latency. The high lateral usage shows that packets actively use shifts along the Y or Z axis to bypass the concentrated X -direction bottleneck. This suggests that the additional lateral choices are particularly useful for traffic dominated by movement along the X axis.

The result for Diagonal-3D is more interesting. Under such highly symmetric traffic, the (4, 4) twist performs fairly well. This makes sense because the original paper proposed this topology to make an asymmetric network behave more like a symmetric one. Nevertheless, FlexRoute significantly recovers the performance of AdaptiveRoute on (3, 4) before its saturation knee, bringing it close to the (4, 4) result. The lateral-usage curve confirms that this recovery coincides with active use of the additional equal-distance choices.

C.3 One more thing: An interesting question is whether we can apply FlexRoute to the (4, 4) torus to achieve even better performance. In fact, it cannot provide further improvement because this topology has no lateral candidates; Appendix C gives a proof by coloring. We also perform experiments showing that the (4, 4) twisted 3D torus with FlexRoute has exactly the same performance as with AdaptiveRoute. The results are also shown in Appendix C.

V. CONCLUSION

In this project, we jointly explore topology and routing design for asymmetric 3D tori. We implement configurable twisted 3D torus topologies in gem5 and show that, for a

$2k \times k \times k$ torus, the $(k-1, k)$ twist achieves the minimum possible diameter among the considered dual twists. However, a shorter path does not necessarily provide more routing choices. To address the reduced shortest path diversity of the (3, 4) topology, we propose FlexRoute, which supplements deadlock-free adaptive minimal routing with at most one congestion-aware lateral hop.

We evaluate on the $8 \times 4 \times 4$ case. The (3, 4) topology reduces latency and improves reception rate, while FlexRoute further releases the performance potential of the (3, 4) twist under several permutation and directional traffic patterns. We also compare (3, 4) topology with PTT and PDTT to analyze their pros and cons.

VI. LABOR DIVISION

Fan Zhengyang: Topology design, analysis and implementation; part of routing algorithm design and implementation; evaluation; and report.

Xu Yudong: Part of routing algorithm design and implementation.

REFERENCES

- [1] J. M. Cámara, M. Moretó, E. Vallejo, R. Beivide, J. Miguel-Alonso, C. Martínez, and J. Navaridas, “Twisted torus topologies for enhanced interconnection networks,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 21, no. 12, pp. 1765–1778, Dec. 2010, doi: 10.1109/TPDS.2010.30.
- [2] W. J. Dally and B. Towles, *Principles and Practices of Interconnection Networks*. Morgan Kaufmann, 2004.
- [3] Google Cloud, “TPU v4,” Cloud TPU Documentation. [Online]. Available: <https://cloud.google.com/tpu/docs/v4>.

APPENDIX A
PROOF OF THEOREM 1

Define the cyclic distance

$$\rho_n(r) = \min_{\ell \in \mathbb{Z}} |r + n\ell|. \quad (15)$$

By translation invariance, it suffices to consider the distance from the origin to (x, y, z) . If a path winds u and v times around the Y and Z boundaries, respectively, its exact minimum length is

$$d_{p,q}(x, y, z) = \min_{u,v \in \mathbb{Z}} \{|y + ku| + |z + kv| + \rho_{2k}(x + pu + qv)\}. \quad (16)$$

Reversing either short dimension replaces a shift by its additive inverse, and exchanging Y and Z exchanges p and q . Hence, without loss of generality, let $0 \leq p \leq q \leq k$.

We first prove the universal lower bound. Suppose $k = 2m$ and, for contradiction, that $D(p, q) \leq 3m - 2$. On the central slice (x, m, m) , only $(u, v) \in \{0, -1\}^2$ can satisfy this bound. All four choices spend $2m$ short-dimension hops, so the four cyclic balls

$$B(0, m-2), B(p, m-2), B(q, m-2), B(p+q, m-2) \quad (17)$$

must cover $\mathbb{Z}_{4m}$. Two radius- h balls can cover the discrete gap between their centers only if the gap is at most $2h + 1$. Applying this to the first and last gaps between the ordered centers in Eq. (17) gives

$$p \leq 2m - 3, \quad p + q \geq 2m + 3. \quad (18)$$

Write $p = 2a + e$ and $q = 2b + f$, where $e, f \in \{0, 1\}$. Equation (18) implies $1 \leq a \leq b$, $a \leq m - 2$, and $a + b \geq m + 1$. Consider

$$(x, y, z) = (a + b - m + 1, 2m - b - 1, 2m - a - 1). \quad (19)$$

Substitution into Eq. (16) gives, for $(u, v) = (0, 0), (-1, 0), (0, -1), (-1, -1)$, respectively,

$$\begin{aligned} (C_{00}, C_{10}) &= (3m - 1, 3m - 1 + e), \\ (C_{01}, C_{11}) &= (3m - 1 + f, 3m + 3 - e - f). \end{aligned} \quad (20)$$

Every component is at least $3m - 1$. Any other winding has short-dimension cost at least $3m + 1$, so the node in Eq. (19) is at distance at least $3m - 1$, contradicting the assumption. Thus

$$D(4m, 2m, 2m; p, q) \geq 3m - 1. \quad (21)$$

Now let $k = 2m + 1$ and suppose $D(p, q) \leq 3m$. On the slice (x, m, m) , the same four windings yield the balls

$$B(0, m), B(p, m - 1), B(q, m - 1), B(p + q, m - 2) \quad (22)$$

in $\mathbb{Z}_{4m+2}$. If $p + q \leq 2m + 2$, the point $x = 3m + 1$ is outside all four balls. Moreover, $p = 2m + 1$ forces $p = q = 2m + 1$, in which case $x = m + 1$ is uncovered. Consequently,

$$p + q \geq 2m + 3, \quad 2 \leq p \leq 2m. \quad (23)$$

Again write $p = 2a + e$ and $q = 2b + f$. Then $1 \leq a \leq b \leq m$ and $a + b \geq m + 1$. For the node

$$(x, y, z) = (a + b - m + ef, 2m + 1 - b, 2m + 1 - a), \quad (24)$$

the four low-winding costs are

$$\begin{aligned} C_{00} &= 3m + 2 + ef, & C_{10} &= 3m + 1 + e - ef, \\ C_{01} &= 3m + 1 + f - ef, & C_{11} &= 3m + 2 - e - f + ef. \end{aligned} \quad (25)$$

They are all at least $3m + 1$. Every other winding has short-dimension cost at least $k + a + b \geq 3m + 2$. Hence

$$D(4m + 2, 2m + 1, 2m + 1; p, q) \geq 3m + 1. \quad (26)$$

For $k = 1$, the two-node graph has diameter one. For $k = 2$, each node has at most six distinct neighbors among the other seven nodes, so the graph is not complete and its diameter is at least two. Equations (21) and (26), together with the direct $k = 1$ and $k = 2$ cases, establish

$$D(2k, k, k; p, q) \geq \left\lceil \frac{3k}{2} \right\rceil - 1 \quad (27)$$

for every dual twist (p, q) .

It remains to show that $(p, q) = (k - 1, k)$ attains this bound. For any $0 \leq y, z < k$, consider the four paths corresponding to $(u, v) \in \{0, -1\}^2$:

$$\begin{aligned} C_{00} &= y + z + \rho_{2k}(x), \\ C_{10} &= k - y + z + \rho_{2k}(x - k + 1), \\ C_{01} &= y + k - z + \rho_{2k}(x - k), \\ C_{11} &= 2k - y - z + \rho_{2k}(x + 1). \end{aligned} \quad (28)$$

Because $\rho_{2k}(r) + \rho_{2k}(r - k) = k$, these lengths satisfy

$$C_{00} + C_{10} + C_{01} + C_{11} = 6k. \quad (29)$$

For odd k , Eq. (29) implies $\min C_{ij} \leq \lfloor 3k/2 \rfloor = (3k - 1)/2$. For even k , the average is $3k/2$, but C_{00} and C_{10} have opposite parity; thus the four integers cannot all equal their average, and $\min C_{ij} \leq 3k/2 - 1$. Therefore

$$D(2k, k, k; k - 1, k) \leq \left\lceil \frac{3k}{2} \right\rceil - 1. \quad (30)$$

Combining Eqs. (27) and (30) proves Theorem 1.

APPENDIX B

ENUMERATION OF AVERAGE SHORTEST-PATH DISTANCE

Because every topology considered here is vertex-transitive, every source has the same distance distribution. It is therefore sufficient to fix the source at $(0, 0, 0)$ and enumerate all 128 destinations, including the source itself whose distance is zero.

For a twist (p, q) , let $L_h(p, q)$ be the set of routers whose shortest distance from $(0, 0, 0)$ is exactly h , and let $N_h(p, q) = |L_h(p, q)|$. We start with $L_0 = \{(0, 0, 0)\}$ and repeatedly construct the next layer as

$$L_{h+1} = \{u \mid u \text{ is adjacent to some } v \in L_h\} \setminus \bigcup_{i=0}^h L_i. \quad (31)$$

Adjacency is determined directly by Eqs. (1)–(5). Thus, this enumeration includes both ordinary links and the twisted wraparound links. Once all 128 routers have been assigned to a layer, the average distance, including the zero-distance source, is

$$\bar{d}(p, q) = \frac{1}{128} \sum_{h=1}^{D(p, q)} h N_h(p, q). \quad (32)$$

For the ordinary $(0, 0)$ torus, the layer sizes from hop 0 through hop 8 are

$$(N_0, \dots, N_8) = (1, 6, 16, 26, 30, 26, 16, 6, 1). \quad (33)$$

Therefore,

$$\begin{aligned} \bar{d}(0, 0) &= \frac{1}{128} [1(6) + 2(16) + 3(26) + 4(30) \\ &\quad + 5(26) + 6(16) + 7(6) + 8(1)] \\ &= \frac{512}{128} = 4.0000. \end{aligned} \quad (34)$$

For PTT with $(p, q) = (4, 0)$, the layer sizes from hop 0 through hop 6 are

$$(N_0, \dots, N_6) = (1, 6, 17, 32, 39, 26, 7), \quad (35)$$

which gives

$$\begin{aligned} \bar{d}(4, 0) &= \frac{1}{128} [1(6) + 2(17) + 3(32) \\ &\quad + 4(39) + 5(26) + 6(7)] \\ &= \frac{464}{128} = 3.6250. \end{aligned} \quad (36)$$

For PDTT with $(p, q) = (4, 4)$, the layer sizes are

$$(N_0, \dots, N_6) = (1, 6, 18, 38, 43, 20, 2), \quad (37)$$

and hence

$$\begin{aligned} \bar{d}(4, 4) &= \frac{1}{128} [1(6) + 2(18) + 3(38) \\ &\quad + 4(43) + 5(20) + 6(2)] \\ &= \frac{440}{128} = 3.4375. \end{aligned} \quad (38)$$

Finally, for our $(3, 4)$ twist, all routers are reached within five hops and the layer sizes are

$$(N_0, \dots, N_5) = (1, 6, 18, 38, 49, 16). \quad (39)$$

Thus,

$$\begin{aligned} \bar{d}(3, 4) &= \frac{1}{128} [1(6) + 2(18) + 3(38) \\ &\quad + 4(49) + 5(16)] \\ &= \frac{432}{128} = 3.3750. \end{aligned} \quad (40)$$

In each case, the largest index with a nonempty layer also gives the diameter reported in Table I.

APPENDIX C

WHY THE $(4, 4)$ TWIST HAS NO LATERAL CANDIDATES

Color each router of the $8 \times 4 \times 4$ $(4, 4)$ twisted torus by

$$\chi(x, y, z) = (x + y + z) \bmod 2. \quad (41)$$

Every link joins routers of opposite colors. An ordinary link changes one coordinate by one; an X wraparound changes x by seven; and a twisted Y or Z wraparound changes the coordinate sum by $4 - 3 = 1$ modulo two. Thus, Eq. (41) is a valid bipartite coloring.

For any fixed destination t , the parity of every path from a router v to t is fixed by the colors of its endpoints. Consequently,

$$d(v, t) \equiv \chi(v) - \chi(t) \pmod{2}. \quad (42)$$

If u is adjacent to v , then $\chi(u) \neq \chi(v)$, so $d(u, t)$ and $d(v, t)$ have opposite parity and cannot be equal. Therefore

$$S(v, t) = \emptyset \quad \text{for every } v \neq t \quad (43)$$

on the $(4, 4)$ topology, and FlexRoute has no lateral hop to select.

This property does not hold for the $(3, 4)$ twist. For example, with $t = (0, 0, 0)$, routers $v = (3, 0, 0)$ and $u = (0, 3, 0)$ are adjacent across the twisted Y boundary, while

$$d(v, t) = d(u, t) = 3. \quad (44)$$

Hence $u \in S(v, t)$, showing why FlexRoute can provide an additional routing choice on $(3, 4)$ but not on $(4, 4)$.

We further compare AdaptiveRoute and FlexRoute on the $(4, 4)$ topology under uniform-random, Bit-Complement, Bit-Reversal, Perfect-Shuffle, Diagonal-3D, X-Tornado, and Y-Tornado traffic. Each traffic pattern is swept from injection rate 0.025 to 1.000 in increments of 0.025. As shown in Fig. 8, the two routing policies overlap at every point. Across all 280 cases, their reception rate, queueing latency, network latency, total latency, and average hop count are exactly equal. This provides experimental confirmation that FlexRoute degenerates to AdaptiveRoute when $S(v, t)$ is always empty.

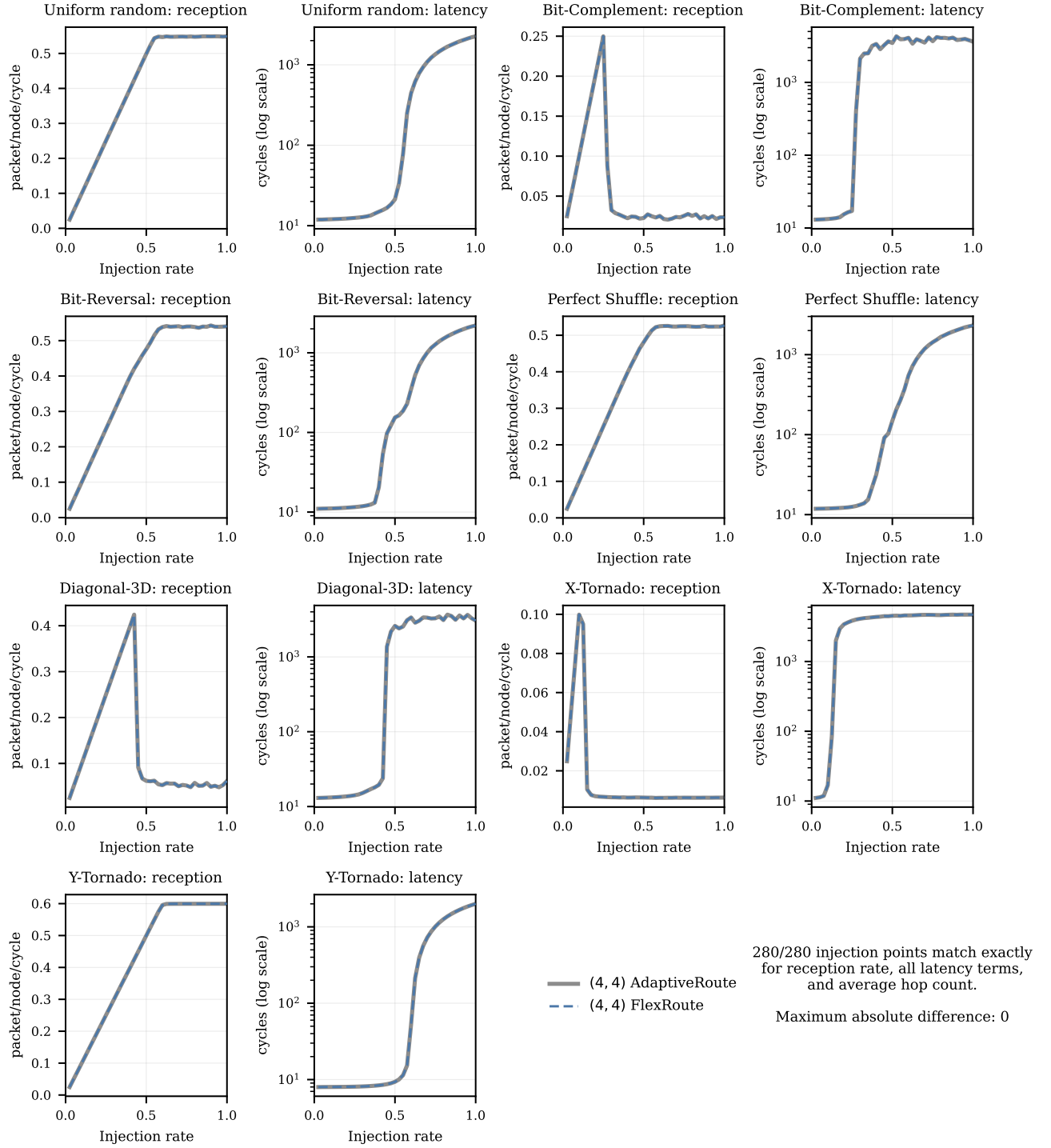


Fig. 8. AdaptiveRoute and FlexRoute on the $(4, 4)$ twisted torus under seven traffic patterns. The solid and dashed curves overlap exactly for both reception rate and total latency at all 280 injection points.